AIoT PROJECT · SEMESTER VII

FROSTMIND

An AI-driven cold-chain monitoring and intervention system

Project Documentation

Phase 7: Deployment, Documentation & Presentation

Prepared by	Team FROSTMIND
Document version	1.0 (27 October 2026)
Code base	github.com/UzairManjre/frostmind, branch feature/load-planner-models-4-5
Evaluation focus	Deployment, usability, documentation quality, demonstration, presentation, future scope, maintenance strategy

Contents

1. Executive summary 4
2. Problem statement and objectives 5
2.1 The problem 5
2.2 Objectives 5
2.3 Scope 5
3. System architecture 6
3.1 How one reading travels 6
3.2 Technology stack 6
4. Hardware 8
4.1 Bill of materials 8
4.2 Pin map and wiring 8
4.3 Power 9
5. Firmware (ESP32-S3) 10
5.1 Sensing and the on-device rule (Model 0) 10
5.2 Automatic AC pairing by thermal feedback 10
5.3 Bluetooth link to the phone 10
5.4 Status light strip 11
5.5 Bench tools 11
6. Phone bridge app 12
6.1 The six-step setup wizard 12
6.2 Live dashboard 12
6.3 Reliability on a moving truck 13
7. Backend 14
7.1 Data model 14
7.2 The ingest pipeline 14
7.3 Security 14
8. Admin panel 15
9. AI and machine-learning layer 17
9.1 Model 0: on-device rule 17
9.2 Model 1: anomaly detection 17
9.3 Model 2: root-cause classification (deferred) 18
9.4 Model 3: shelf life (Arrhenius kinetics) 18
9.5 Model 4: mixed-load setpoint 18
9.6 Model 5: which products can ship together 19
9.7 Data collection and cleaning 20
9.8 The local LLM layer 20
10. Deployment 21
10.1 Ports 21

10.2 Prerequisites 21
10.3 Installing on a fresh laptop 21
10.4 No IP lock: it works on any network 22
10.5 First-time phone setup 22
10.6 Configuration 22
10.7 Flashing the firmware 22
10.8 Troubleshooting 23
10.9 Path to production 23
11. Usability 24
11.1 Users and their goals 24
11.2 Design principles applied 24
11.3 Improvements found through real use 24
12. Testing and quality assurance 26
13. Demonstration plan 27
13.1 Before the demo 27
13.2 Ten-minute script 27
13.3 Fallbacks 27
14. Future scope 28
15. Maintenance strategy 29
15.1 Routine tasks 29
15.2 Model governance 29
15.3 Monitoring and incident response 30
15.4 Data and schema changes 30
15.5 Roles 30
16. Limitations 31
17. Conclusion 31
18. References 32
Appendix A. Repository structure 33
Appendix B. API endpoints 34
Appendix C. Command cheat sheet 35
Appendix D. Glossary 35

1. Executive summary

Refrigerated shipments of food and medicines fail quietly: a door left open, a cooling unit that stops, a load packed with products that should never travel together. Most low-cost data loggers only *record* the temperature. Someone reads the log after the damage is done.

FROSTMIND senses, acts and explains. A small ESP32 box on the truck reads temperature, humidity and the door. It decides on its own when something is wrong and fires an infrared command at the cooling unit. Through the driver's phone it streams every reading to a local server, where the AI layer computes how much shelf life each product has used, recommends the right temperature for mixed loads, and checks which products can share a compartment. A local language model then explains every incident in plain English. Nothing leaves the laptop: no cloud account is needed and no data is sent anywhere.

What was delivered	Evidence
Working hardware	Seeed XIAO ESP32-S3 with DHT22, door reed switch, IR transmitter and an 11-LED status light strip. Flashed and verified on the bench (24.5 °C, door open/closed, strip in GRB order).
Complete software stack	Firmware, Android phone app (PWA), FastAPI backend and Next.js admin panel. **72 automated backend tests pass**; the web app passes lint, type checks and a production build; the firmware compiles with zero warnings.
Seven AI components	Model 0 on-device rule (live), Model 1 anomaly detector (validated), Model 2 root-cause classifier (deferred, hardware-gated), Model 3 Arrhenius shelf life (live), **Model 4 mixed-load setpoint: MAE 0.16 °C**, **Model 5 co-shipment compatibility: ROC-AUC 0.930**, local LLM explanations (live).
One-command deployment	`python start_frostmind.py` runs the whole system on any laptop. It finds the network address itself, so nothing is tied to a fixed IP.
Usable by a driver	A 6-step guided setup, no typing of addresses or device IDs, plain-language error hints, AI explanations with a *Do:* action, and a light strip that shows the state of the trip at a glance.
Documentation and presentation	This document, a slide deck, wiring and BOM guide, firmware and phone READMEs, API docs at `/docs`, and the Model 4/5 technical report.

Table 1. *Phase 7 deliverables at a glance*

The rest of this document follows the Phase 7 evaluation criteria. Sections 2 to 9 document the system itself. Section 10 covers deployment, 11 usability, 12 testing, 13 the demonstration plan, 14 future scope and 15 the maintenance strategy.

2. Problem statement and objectives

2.1 The problem

Cold-chain products have a temperature window they must stay inside, from vaccines (2 to 8 °C) to frozen food (−18 °C or colder). An excursion does not destroy a product instantly. Damage accumulates with time and temperature, so the questions that matter in practice are:

- **Is something going wrong right now**, and can it be corrected before the product is harmed?
- **How much damage has already been done?** A 20-minute excursion and a 6-hour one are very different.
- **What should this particular load be kept at?** A mixed load of apples, pears and lettuce has no single label temperature.
- **Which products can safely share a compartment?** Ripening bananas give off ethylene, which spoils lettuce; medicines must never travel with food.
- **What happened, in words a manager or auditor can act on?**

2.2 Objectives

1. Monitor temperature, humidity and door state on a refrigerated load every 2 seconds with low-cost hardware.
2. React automatically on the truck: correct the cooling unit over infrared without waiting for a human or an internet connection.
3. Quantify product damage with a physics-based shelf-life model, per product.
4. Use machine learning where real data supports it: mixed-load setpoints (Model 4) and co-shipment compatibility (Model 5).
5. Explain every incident in plain language with a local LLM that can never take a physical action itself.
6. Make it deployable on any laptop and usable by a driver with no technical training.

2.3 Scope

In scope: one truck or compartment per device, an Android phone as the network bridge, a laptop as the server, and a room air conditioner as the stand-in for a reefer unit during testing. Out of scope for this phase: cellular or satellite connectivity, a cloud deployment, iOS support (Safari does not implement Web Bluetooth), and certification for regulated pharmaceutical decisions.

3. System architecture

FROSTMIND has four tiers. The **edge device** senses and acts. The **phone bridge** carries data between Bluetooth and the network, because the ESP32 box has no internet radio of its own. The **backend** stores readings and runs the AI models. The **admin panel** is the fleet manager's view. A **local LLM** sits beside the backend and writes the explanations.

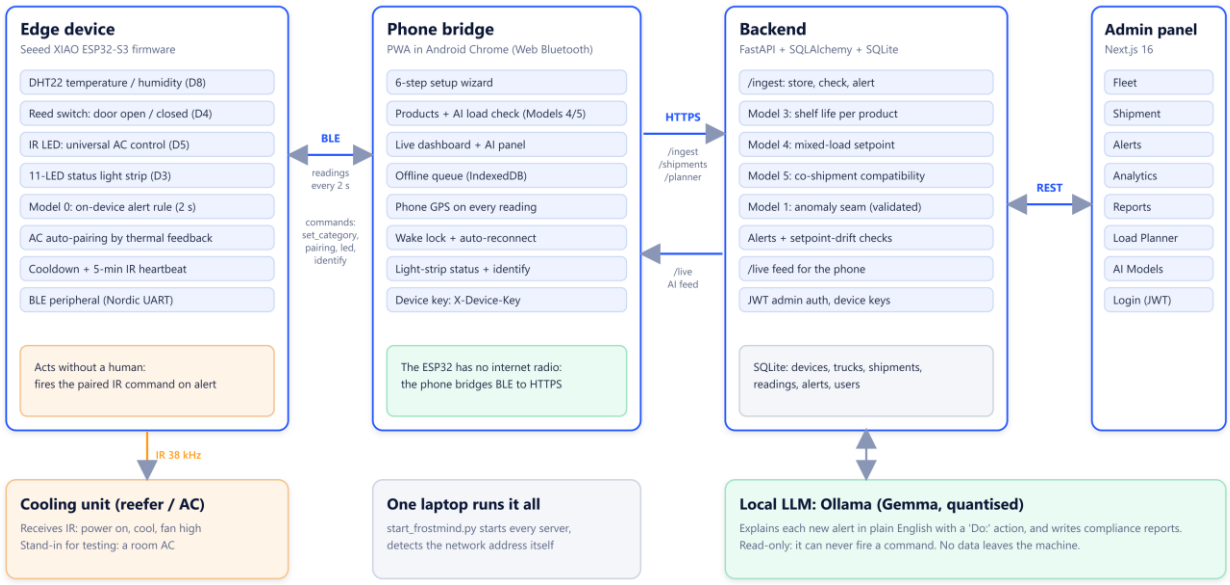


Figure 1. *System architecture: edge device, phone bridge, backend, admin panel and the local LLM*

3.1 How one reading travels

1. The ESP32 samples the DHT22 and the reed switch every 2 seconds and applies the Model 0 rule. If the rule trips and an AC protocol is paired, it fires the IR command immediately (5-minute cooldown).
2. The reading goes to the phone over Bluetooth Low Energy as one line of JSON.
3. The phone adds its GPS position and puts the reading in an offline queue (IndexedDB), so nothing is lost when the network drops.
4. The sync manager uploads queued readings to `POST /ingest` over HTTPS, authenticated with the device's own key.
5. The backend stores the readings, updates Model 3 shelf life for every product, checks the Model 4 setpoint, and raises alerts.
6. For the newest alert of each type, a background task asks the local LLM for an explanation and stores it on the alert.
7. The phone dashboard (every 10 s) and the admin panel show the readings, shelf life, alerts and AI explanations. The phone also sends the shelf life and alert level back to the box's light strip.

3.2 Technology stack

Tier	Technology	Why
Edge device	Seeed XIAO ESP32-S3, Arduino core 3.3.11, DHT sensor library, IRremoteESP8266 2.9.0, ArduinoJson 7, bundled BLE, RMT for the LED strip	Small, cheap, Bluetooth built in; IRremoteESP8266 speaks about 65 AC protocols

Tier	Technology	Why
Phone bridge	Plain HTML/JS ES modules (no build step), Web Bluetooth, IndexedDB, service worker, Geolocation, Wake Lock	Runs in Chrome on any Android phone with nothing to install; works offline
Backend	Python 3.12, FastAPI, SQLAlchemy 2, SQLite, Pydantic 2, PyJWT, scikit-learn, NumPy/SciPy	Fast to build, typed API with automatic docs, ORM ready for PostgreSQL
Admin panel	Next.js 16, React 19, TypeScript, dependency-free CSS design system	Modern web UI; served from the same laptop
AI layer	scikit-learn (Random Forest, Histogram Gradient Boosting, Isolation Forest), physics models in NumPy/SciPy	Interpretable models that run on a laptop CPU
LLM	Ollama with a quantised Gemma instruction model (QAT, 4-bit)	Runs locally: no cloud API, no per-call cost, no data leaving the machine

Table 2. *Technology stack*

4. Hardware

4.1 Bill of materials

Part	Specification	Qty	Purpose
Seeed XIAO ESP32-S3	ESP32-S3R8, 8 MB PSRAM, USB-C, BLE 5	1	Brain of the device; powered and flashed over USB-C
DHT22 (AM2302) module	3-pin single-wire, ±0.5 °C, 0-100 % RH	1	Temperature and humidity
MC-38 reed switch	2-wire, normally open, with magnet	1	Door open/closed
IR LED	940 nm, 5 mm	1	Sends AC commands (transmit only, no IR receiver in this build)
WS2812 / WS281x LED strip	5 V addressable RGB, GRB colour order, 11 LEDs	1	Status light: pairing, shelf life, alerts
Resistor (optional)	100-220 Ω	1	IR LED current limit
Breadboard, jumper wires	-	-	Connections; one ground rail shared by all parts
USB-C cable	Data-capable	1	Power and flashing (a charge-only cable cannot flash)

Table 3. *Bill of materials*

4.2 Pin map and wiring

> **Important**
> Wire by the **D-label printed next to each hole**. On the XIAO ESP32-S3, D*n* is **not** GPIO *n*: D3 = GPIO4, D4 = GPIO5, D5 = GPIO6, D6 = GPIO43, D8 = GPIO7. The XIAO ESP32-C3 looks identical but maps differently, so its pinout does not apply. This was the cause of an early 'IR LED never lights' problem.

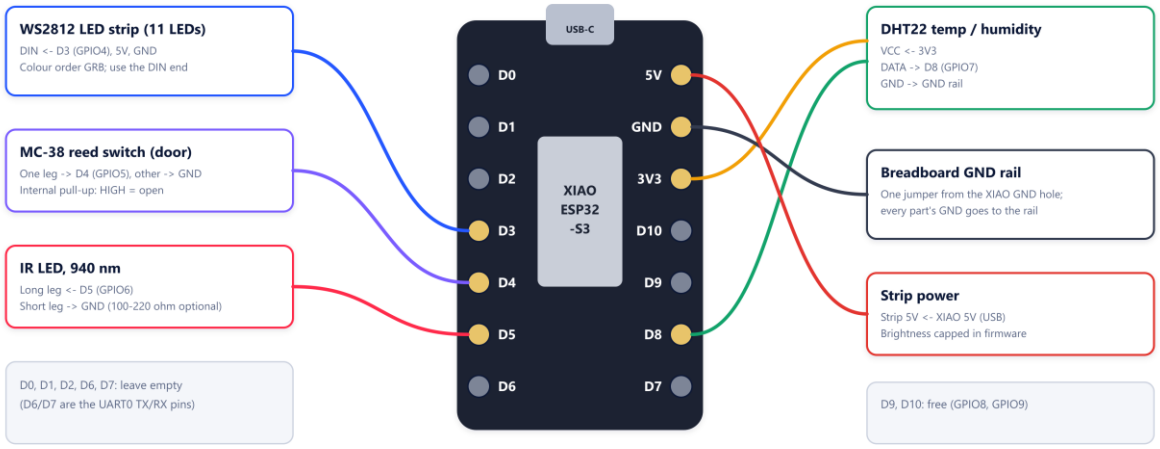


Figure 2. *Wiring on the XIAO ESP32-S3*

XIAO pad	GPIO	Function	Connects to
3V3	-	3.3 V supply	DHT22 VCC

XIAO pad	GPIO	Function	Connects to
5V	-	5 V from USB	LED strip 5V
GND	-	Ground	Breadboard GND rail: DHT GND, reed switch, IR LED short leg, strip GND
D8	GPIO7	Single-wire data	DHT22 DATA
D4	GPIO5	Input, internal pull-up	Reed switch (other leg to GND). HIGH = door open
D5	GPIO6	Digital output	IR LED long leg (anode)
D3	GPIO4	Output (RMT peripheral)	LED strip DIN (use the strip's input end)
D0-D2, D6, D7, D9, D10	-	Free	Leave empty (D6/D7 are the UART0 pins)

Table 4. *Pin map (constants in firmware/frostmind_main/frostmind_main.ino)*

4.3 Power

Everything runs from the USB-C port. The firmware caps the strip's brightness (80/255 after gamma correction), so 11 LEDs stay well under the USB budget. On a truck the same 5 V would come from a USB adapter on the vehicle supply, with a battery backup listed as future work.

5. Firmware (ESP32-S3)

One sketch, `firmware/frostmind_main/frostmind_main.ino`, has five responsibilities: sensing and the on-device rule, AC control, automatic AC pairing, the Bluetooth link to the phone, and the status light strip.

5.1 Sensing and the on-device rule (Model 0)

Every 2 seconds the firmware reads the DHT22 and the reed switch, keeps the last 6 temperatures, and computes the least-squares slope in °C per minute. The rule is deliberately simple and explainable:

```
rate      = least-squares slope of the last 6 samples      (°C/min)
tempAlert = |rate| > 3.0   OR   |temp - category setpoint| > 3.0 °C
doorAlert = door open for more than 60 s
alert     = tempAlert OR doorAlert
```

On an alert, if an AC protocol is paired, the device sends power on, cool, fan high over IR, then waits at least 5 minutes before it can fire again. Every 5 minutes it also re-sends the last command (the **heartbeat**). With no IR receiver it cannot confirm that a command landed, so re-asserting the intended state is the only mitigation. The heartbeat also shows the phone that the IR path is alive.

5.2 Automatic AC pairing by thermal feedback

Normally a universal remote learns a protocol by capturing the original remote with an IR receiver. This build has none, so FROSTMIND uses the room itself as the feedback signal:

1. Measure the baseline temperature slope for about 60 s without touching the AC.
2. For each candidate protocol (a shortlist of common brands first, Samsung first, then all of the library's roughly 65 protocols), send power on + cool and measure the slope for about 100 s.
3. If the room cools noticeably faster than the baseline, send power off on the same protocol and check that the trend reverses within about 45 s. A coincidental drop is rejected here.
4. On success, save the protocol in flash (NVS), so it survives reboots. If nothing matches, a manual mode lets a person step through candidates from the phone or the serial console.

5.3 Bluetooth link to the phone

The device is a BLE peripheral using the Nordic UART Service pattern. Messages are newline-delimited JSON in both directions. The phone writes commands in 20-byte chunks, which the firmware reassembles before parsing.

Direction	Messages
Device to phone, every 2 s	`{"ts", "temp", "hum", "door", "roc", "alert", "ac_commanded", "ac_degrees", "ac_power", "ac_last_cmd_ms"}`
Device to phone, on change	`{"type":"pairing_status", "state": scanning | matched | no_match | idle, "candidate_index", "total"}`
Phone to device	`set_category`, `start_pairing`, `pairing_status`, `manual_pair`, `manual_confirm`, `led` (shelf life left + alert level for the strip), `identify` (5 s rainbow)

Table 5. *BLE contract summary (full contract: firmware/frostmind_main/README.md)*

5.4 Status light strip

An 11-LED WS2812 strip gives the driver the state of the trip at a glance. It is driven straight from the ESP32's RMT peripheral, so no extra library is needed and the timing is done in hardware. Frames are drawn in floating point at 60 fps, lights move at sub-pixel positions, every change cross-fades over 450 ms, and the output is gamma-corrected and dithered.

The strip follows a **workflow** and shows **one look at a time**. Nothing is layered on top of anything else, and short animations queue up instead of cutting each other off. Problems are only shown while a phone is connected and a trip is live: with no phone connected nobody would see them, and during setup no product or setpoint has been chosen yet.

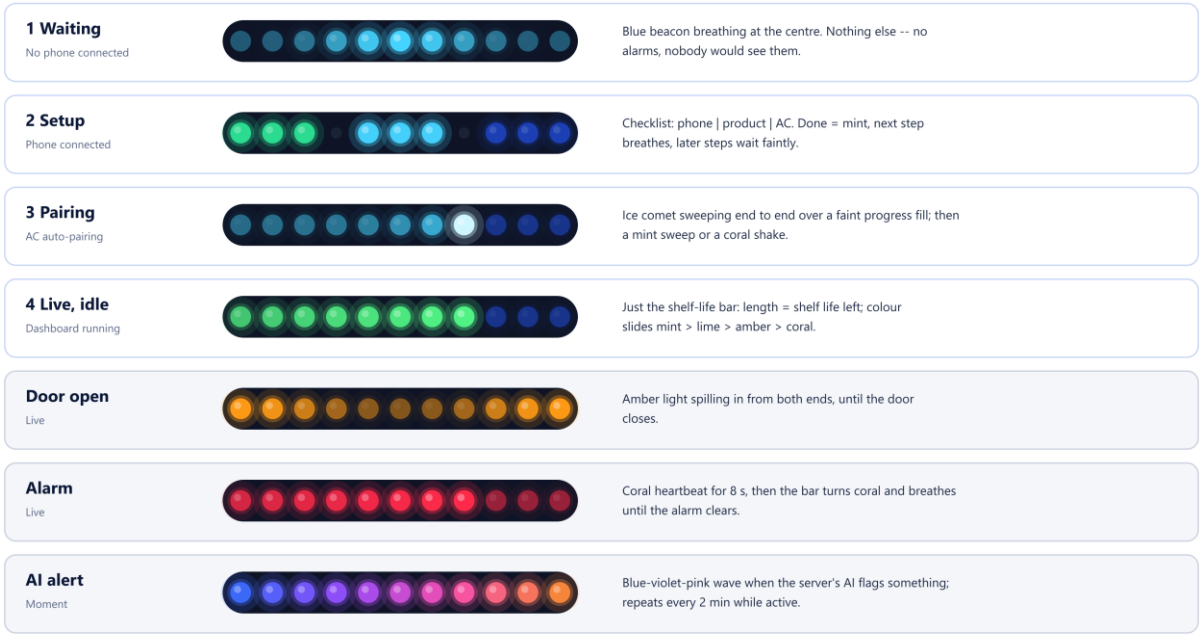


Figure 3. *The light strip's workflow states*

5.5 Bench tools

`scripts/esp_console.py` is a live serial console that formats each reading and passes keys to the firmware:

Key	Action
x	IR LED test: 3 s of flashing (view it through a phone camera; IR is invisible to the eye)
g	Light strip colour test: red, green, blue, white for 1 s each (checks the GRB colour order)
d	Light strip demo: the whole workflow in order, 4.5 s per look, with captions
a	Start automatic AC pairing
l, n, digits + Enter, y	Manual pairing: list protocols, next candidate, jump to an index, confirm and save
+, -, f, p, r, c	AC temperature up/down, fan, power, resend, clear the saved pairing

Table 6. *Serial console keys*

6. Phone bridge app

The phone app (`phone-bridge/`) is a Progressive Web App opened in Chrome on Android. It needs HTTPS for Web Bluetooth, which the launcher provides with a self-signed certificate. There is nothing to install from an app store.

6.1 The six-step setup wizard

#	Step	What happens
1	Pair phone to device	Chrome's Bluetooth picker lists FROSTMIND boxes (matched on the advertised service or the name). Errors come back as instructions, e.g. 'Bluetooth is off on this phone, turn it on and tap Pair again'.
2	Register device	The device ID is read from the Bluetooth connection, not typed. The device and its truck are registered together, and the device receives its own key.
3	What are you shipping?	Add products and kilograms (search or presets), then **Check load (AI)**. Model 5 says what can travel together and why not; Model 4 gives each compartment's temperature. Create the shipment for this truck's compartment. The box receives the nearest preset and ticks the 'product' step on its light strip.
4	Auto-pair the AC	Starts the thermal-feedback sweep and shows its progress, with manual pairing as a fallback.
5	Confirm live data	Waits for real readings before continuing (device to phone). **Flash the device's lights** checks the other direction: the box plays a rainbow.
6	Go live	Starts the upload queue, GPS and the live dashboard. The light strip plays the 'go live' sweep and shows the shelf-life bar.

Table 7. *Setup wizard*

6.2 Live dashboard

- Tiles: temperature, Bluetooth state, humidity, door, readings waiting to sync, shelf life used, GPS, the AC's last commanded setting and when the last IR command was sent.
- **AI panel**, fed by `GET /shipments/{id}/live` every 10 s: Model 4's recommended setpoint against the actual temperature, shelf life per product (Model 3), and the latest alerts with the LLM's assessment and a *Do:* action.
- **Light strip panel**: what the box should be showing right now, an Identify button and a colour legend.
- Reconnect Bluetooth, and Mark delivered / end shipment.

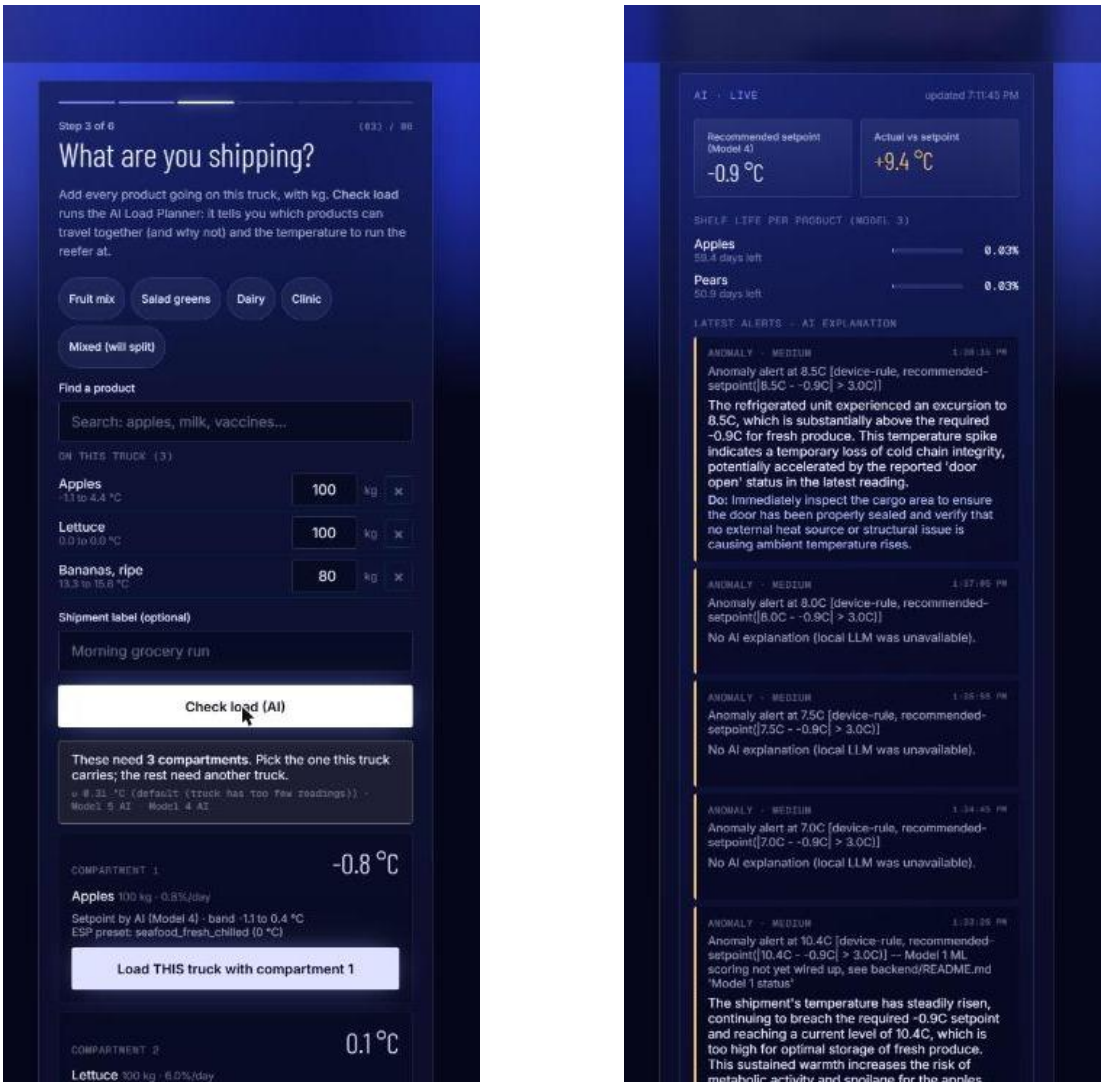


Figure 4. *Left: setup step 3, the AI load check splits products into compartments with setpoints. Right: the dashboard AI panel with setpoint vs actual, per-product shelf life and an AI-explained alert*

6.3 Reliability on a moving truck

- **Offline queue:** readings are stored in IndexedDB first and uploaded when the backend is reachable, so a dead zone loses nothing.
- **Auto-reconnect:** after a Bluetooth drop the app retries with back-off (1, 2, 4, 8, 15, then every 30 s).
- **Screen wake lock:** keeps the tab alive while the dashboard is open. A browser cannot hold Bluetooth in the background; this limit is documented, not hidden.
- **Serialised commands:** Bluetooth writes are queued so two commands never collide.
- **Time handling:** server timestamps are treated as UTC, so alert times are correct in India (UTC+5:30).

7. Backend

The backend (`backend/`) is a FastAPI application over SQLAlchemy and SQLite. It exposes a typed REST API with interactive documentation at `/docs`. The main endpoint groups are `/auth`, `/devices` (registration, pairing status), `/trucks`, `/shipments` (create, update, readings, alerts, live feed, CSV report, compliance report), `/ingest`, `/alerts`, `/analytics/fleet`, `/products`, `/planner/plan`, `/ai/status`, `/ai/llm/test` and `/health`; the full list is in Appendix B.

7.1 Data model

Entity	Holds
Device	MAC/ID, name, provisioning key, last seen, AC pairing state and protocol
Truck	Label, plate, assigned device
Shipment	Category or product list (mixed load), status, start/end, Model 4 setpoint, safe band and method, Model 3 shelf life (shipment level and per product), threshold overrides
SensorReading	Timestamp (server receipt time), temperature, humidity, door, rate of change, device alert flag, anomaly score, latitude/longitude
Alert	Type, severity, message, acknowledged, LLM assessment, recommended action and narrative
User	Admin accounts (hashed passwords) for the panel

Table 8. *Database entities*

Schema changes are additive: on start-up, `ensure_columns()` adds any new nullable column to an existing database, so updating the code never requires deleting data.

7.2 The ingest pipeline

1. Authenticate the device with its `X-Device-Key` and check that the shipment belongs to it.
2. Store the batch, timestamped at the server's receipt time (the device clock is uptime, not wall-clock).
3. Update Model 3 shelf life for every product; the most perishable product drives the shipment's number.
4. Raise alerts: an **anomaly** (device rule tripped, or more than 3 °C from Model 4's recommended setpoint on a mixed load), **door open too long**, and **shelf-life thresholds** (80 % used, then 100 %).
5. Queue an LLM explanation for the newest alert of each type, newest first, so a backlog never queues minutes of LLM work.

7.3 Security

- Admin endpoints need a JWT from `POST /auth/login` (8-hour expiry by default).
- Device endpoints (`/ingest`, `/shipments/{id}/live`) need that device's own key, issued at registration.
- Setup endpoints (device registration, trucks, shipment creation, the planner) are open so the phone wizard works without an admin login. Locking them to a fleet enrolment code is on the roadmap.
- Secrets and the default admin account are environment variables (Section 10.6) and must be changed before any real pilot.
- The LLM is read-only: it writes text onto alerts and can never trigger a device or change a shipment.

8. Admin panel

The admin panel (`web/`) is the fleet manager's view: a Next.js application with a consistent design system. It works from the laptop or from any device on the same network.

Page	What it shows
Fleet (home)	Every truck and device with its current shipment, status and last-seen time; refreshes every 5 s
Shipment detail	Live temperature and humidity chart with flagged readings and door-open shading, the mixed-load AI plan (Model 4 setpoint, per-product shelf life), alerts with AI analysis, CSV export and an LLM compliance report
Alerts	All alerts with severity, acknowledgement and the LLM's assessment and recommended action
Analytics	Fleet trends over a chosen window: alerts by type, and trucks ranked by alerts
Reports	Shipment audit trails: each shipment's record for download
Load Planner	Pick products and quantities, get compartments, setpoints, a compatibility matrix with reasons, and create the shipment for a truck
AI Models	Live status and measured scores of every AI component, where each is used, usage counts, and a one-click LLM test

Table 9. *Admin panel pages*

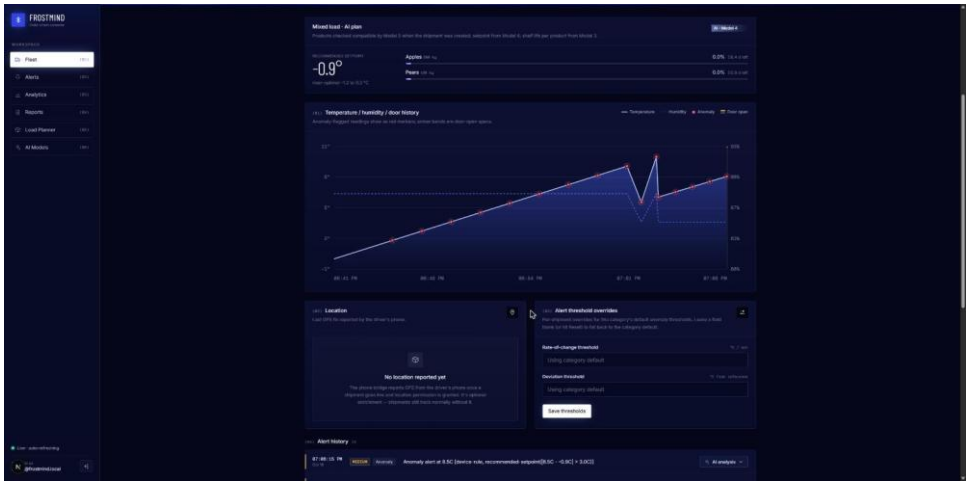


Figure 5. *Shipment page for a mixed load: Model 4 setpoint, per-product shelf life and the live chart*

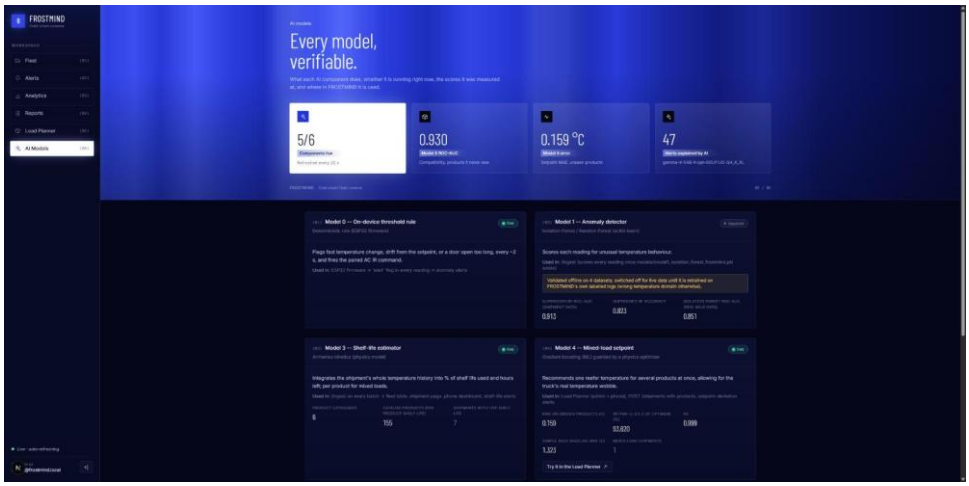


Figure 6. *AI Models page: status, scores and a one-click LLM test*

9. AI and machine-learning layer

FROSTMIND stages its models to the data that actually exists. Rules and physics run where there is no data to learn from yet; machine learning is used where real labelled data exists; the LLM only explains. **Only deterministic logic can trigger a physical action.**

Component	Type	Data and result	Status
Model 0: alert rule	Deterministic rule on the ESP32	Rate of change, setpoint deviation, door time. No training needed.	Live, fires IR
Model 1: anomaly detector	Isolation Forest (unsupervised)	Validated on 4 datasets, ROC-AUC 0.77 to 0.91	Validated, switched off by design
Model 2: root cause	Supervised classifier (planned)	Needs AC current draw (INA219), not sourced	Deferred
Model 3: shelf life	Arrhenius kinetics (physics)	6 categories plus a Q10 model per product	Live, every reading
Model 4: mixed-load setpoint	Physics optimiser + gradient-boosting student	MAE 0.16 °C, 93.6 % within ±0.5 °C on unseen products	Live (guarded)
Model 5: compatibility	Random Forest over 13 pair features	ROC-AUC 0.930, MCC 0.717 on unseen products	Live + policy vetoes
LLM layer	Local Gemma via Ollama	Explains alerts, writes compliance reports	Live, read-only

Table 10. *The AI components*

9.1 Model 0: on-device rule

Described in Section 5.1. It is the only component allowed to act physically, because it is fully predictable and needs no network.

9.2 Model 1: anomaly detection

An Isolation Forest (`contamination=0.02`) trained only on normal data, using rolling mean, rolling standard deviation, rate of change and door-open duration over 10 readings. The decision threshold is chosen on one half of the held-out data and the metrics are reported on the untouched other half.

Dataset	Real or simulated	Accuracy	ROC-AUC	F1
NAB machine temperature	Real	85.6 %	0.770	0.450
Kaggle cold-chain milk (92 sessions)	Real	83.8 %	0.851	0.751
Kaggle shipment failure (semi-supervised)	Simulated	74.2 %	0.809	0.719
Kaggle shipment failure (supervised Random Forest)	Simulated	82.3 %	0.913	0.661

Table 11. *Model 1 validation results*

It is switched off against live data for two stated reasons: the shipment models use trip-level features, not the 2-second stream; and each model is calibrated to one temperature band, so frozen food would look anomalous to a milk-trained model. The backend has a ready seam (`app/ml.py`) that activates it automatically once a model trained on FROSTMIND's own logged sessions exists.

9.3 Model 2: root-cause classification (deferred)

Telling a compressor fault from an open door or a power cut needs the AC's current draw (an INA219 sensor). That sensor was not sourced, and no public dataset exists to simulate it credibly, so no model was faked in its place.

9.4 Model 3: shelf life (Arrhenius kinetics)

```
k(T) = A · exp(−Ea / (R·T))          rate constant, T in kelvin
used fraction = ∫ k(T(t)) dt  /  ( k(T_ref) × rated shelf life )
```

Category	Reference temp	Rated shelf life	Ea (kJ/mol)
Generic refrigerated vaccine	5 °C	~6 months	83
Pasteurised milk	4 °C	~2 weeks	65
Fresh produce	3 °C	~10 days	55
Chilled raw meat/poultry	2 °C	~5 days	60
Frozen food	−18 °C	~6 months	45
Fresh chilled seafood	0 °C	~2 days	70

Table 12. *Model 3 product categories (illustrative literature values, not certified constants)*

For mixed loads each product gets its own shelf-life curve (a Q10 integral from the product catalog), and the most perishable product drives the shipment. Example from the real code: a vaccine shipment held at 5 °C for 48 h with a 6-hour excursion to 12 °C used **1.31 %** of its shelf life.

9.5 Model 4: mixed-load setpoint

Question: one compartment holds several products, so what single temperature is best for all of them? A physics *teacher* minimises the expected quality loss across the products. It accounts for each product's decay rate (Q10), chilling injury below its safe minimum, freezing, and the truck's real temperature wobble σ (measured from real reefer logs), subject to safety constraints (for example medicines 2 to 8 °C, ready-to-eat food at or below 5 °C). A gradient-boosting *student* learned from 16,000 simulated loads and was tested on 4,000 loads made only of products it never saw.

Method	MAE	Within ±0.5 °C	Note
Model 4 student (served)	0.16 °C	93.6 %	R^2 0.999; served only if it agrees with physics to 0.5 °C
Rule baselines (midpoint, weighted mean, max)	1.3-1.4 °C	much lower	Ignore chilling injury and uncertainty

Table 13. *Model 4 results on unseen products*

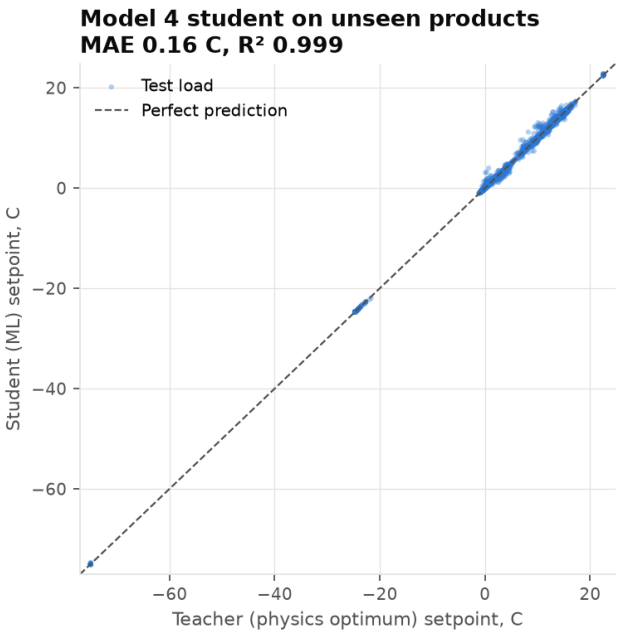


Figure 7. *Model 4: student prediction against the physics optimum on unseen products*

9.6 Model 5: which products can ship together

A Random Forest scores every pair of products on 13 symmetric physical features: temperature-range overlap, humidity overlap, ethylene producer × sensitivity, chilling and freezing risk, odour transfer and more. Labels come from the USDA expert compatibility groups (McGregor, Agricultural Handbook 668), 5,671 produce pairs. It is evaluated on products it never saw, over 30 random splits. Regulatory vetoes are added on top (medicines never with food; raw meat apart from ready-to-eat food), and the order is split into compartments with DSatur graph colouring. Every 'keep apart' comes with a plain-English reason.

Method	ROC-AUC	Accuracy	F1	MCC
Model 5 Random Forest	0.930	86.7 %	0.822	0.717
Best hand-written rule	0.84	-	-	-

Table 14. *Model 5 results on unseen products (mean of 30 product-disjoint splits)*

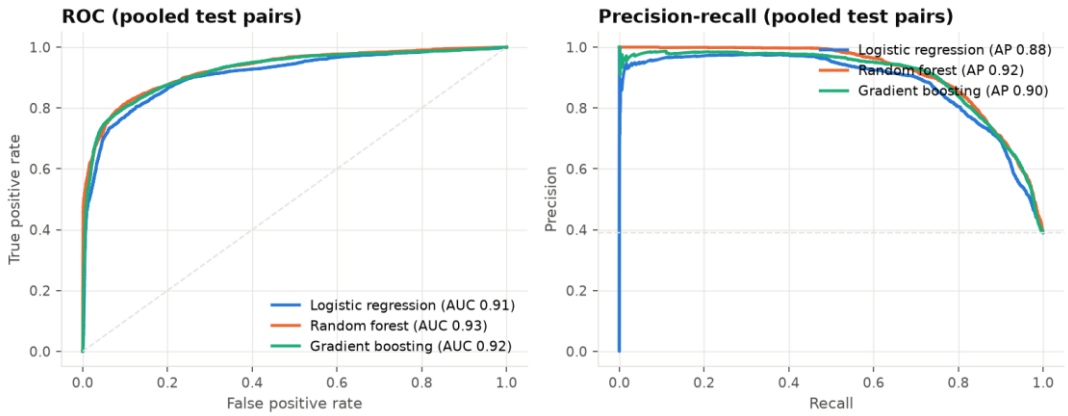


Figure 8. *Model 5 ROC and precision-recall curves*

9.7 Data collection and cleaning

Models 4 and 5 needed data the project did not have, so a product catalog was built from public sources: Engineering ToolBox storage conditions, USDA Handbook 66 (via UMaine Bulletin 4135), the McGregor compatibility groups (FAO reproduction of USDA AH-668), and a cited table for dairy, meat, seafood, frozen food and medicines (USDA FSIS, FDA Food Code, CDC, WHO/EU GDP). Each source was saved as a raw snapshot with its SHA-256 hash. The cleaning pipeline logs every step and its row counts: table extraction and repair, unit conversion (°F to °C), parsing of ranges and storage life, ethylene normalisation, chilling-sensitivity inference, name harmonisation and de-duplication, a cross-source agreement check, gap filling with flags, and validation. The result is **155 products**. Full details are in `FROSTMIND_Model4_Model5_Submission.docx`.

9.8 The local LLM layer

	Role A: alert reasoning	Role B: compliance report
Trigger	Automatically, for the newest alert of each type	On demand from the admin panel
Input	The alert, Model 3 shelf life, Model 4 setpoint, products, recent readings and alerts	The whole shipment history
Output	JSON: assessment, recommended action, narrative	A prose incident narrative for an auditor
Shown on	Phone AI panel and the admin Alerts and Shipment pages	Admin Shipment page

Table 15. *LLM roles*

Reliability rules: every LLM call has a 45-second timeout and returns nothing on any failure, so the deterministic alert always stands on its own. The backend picks the configured model if it is installed, otherwise the smallest installed instruction model. In the end-to-end test an explanation reached the phone **7 seconds** after the alert.

10. Deployment

The whole system runs on one laptop. The driver's phone and any admin browser connect to it over the local Wi-Fi, and the ESP32 talks only to the phone, over Bluetooth.

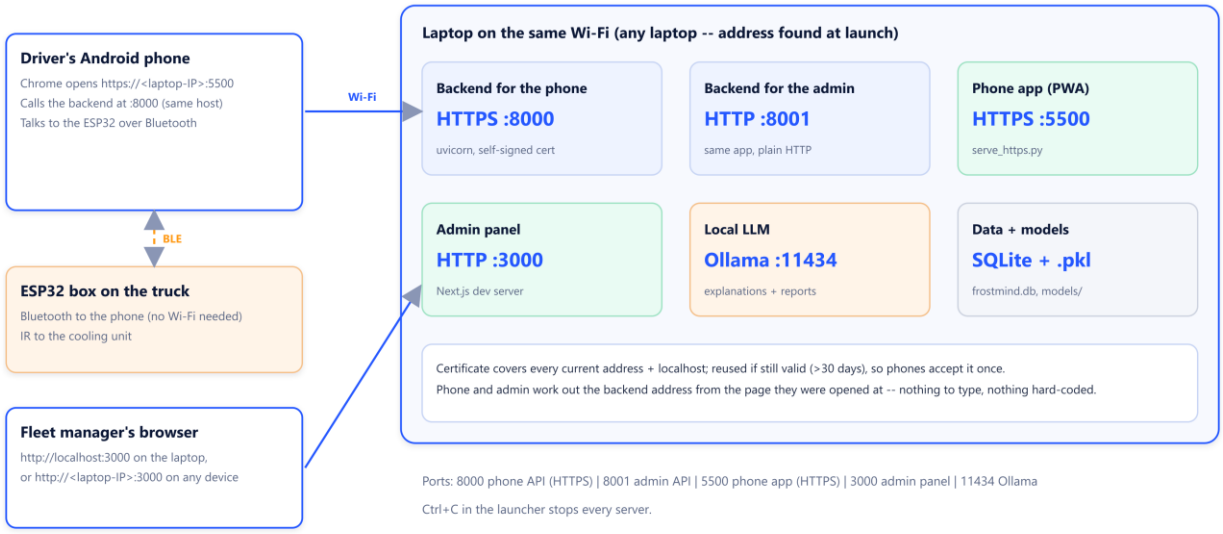


Figure 9. *Deployment topology and ports*

10.1 Ports

Port	Protocol	Service
8000	HTTPS	Backend for the phone (HTTPS, so the HTTPS phone page may call it)
8001	HTTP	Same backend for the admin panel (no certificate prompts)
5500	HTTPS	Phone app static files (`serve_https.py`)
3000	HTTP	Admin panel (Next.js)
11434	HTTP	Ollama, local only

Table 16. *Network ports*

10.2 Prerequisites

- Windows, macOS or Linux laptop with Python 3.12 and Node.js 20 or newer.
- Ollama with the Gemma model (or any small instruction model; the backend picks one automatically).
- Arduino CLI with the esp32 core 3.3.11 and the libraries DHT sensor library, Adafruit Unified Sensor, IRremoteESP8266 2.9.0 and ArduinoJson 7 (only to flash the firmware).
- An Android phone with Chrome, Bluetooth and Location on, on the same Wi-Fi as the laptop.

10.3 Installing on a fresh laptop

```
git clone https://github.com/UzairManjre/frostmind.git && cd frostmind
python -m venv .venv
.venv\Scripts\pip install -r backend/requirements.txt -r scripts/requirements.txt
# build the catalog and train the models (the .pkl files are not in Git)
.venv\Scripts\python scripts/train_all.py
cd web && npm install && cd ..
ollama pull hf.co/unsloth/gemma-4-E4B-it-qat-GGUF:UD-Q4_K_XL
.venv\Scripts\python start_frostmind.py
```

The launcher prints the exact addresses to open, for example `https://192.168.1.20:5500` for the phone and `http://localhost:3000` for the admin panel. Ctrl+C stops every server.

10.4 No IP lock: it works on any network

A hard-coded address breaks the moment the laptop joins a different Wi-Fi. Every part of FROSTMIND works out its addresses at run time instead:

- **Launcher:** detects the laptop's current LAN addresses on every start.
- **Certificate:** generated to cover every current address plus localhost, and reused while it still covers them and has more than 30 days left. A phone accepts it once, not after every restart.
- **Phone app:** calls the backend on the same host it was opened from. A saved address from an old network is ignored automatically.
- **Admin panel:** calls the backend on the same host the page was opened from, and the dev server allows the laptop's current addresses, so it also works from another device.

10.5 First-time phone setup

1. On the phone, open `https://<laptop-IP>:8000/health`, tap Advanced, then Proceed (accepts the certificate for the API).
2. Open `https://<laptop-IP>:5500`, tap Advanced, then Proceed. Optionally add it to the home screen.
3. Follow the setup wizard (Section 6.1). If a previous trip is still saved, use Settings, then Reset wizard.

10.6 Configuration

Environment variable	Default	Purpose
FROSTMIND_DATABASE_URL	sqlite:///backend/frostmind.db	Database (any SQLAlchemy URL, e.g. PostgreSQL)
FROSTMIND_JWT_SECRET	development value	Signs admin tokens. **Change before a pilot.**
FROSTMIND_JWT_EXPIRE_MINUTES	480	Admin session length
FROSTMIND_ADMIN_EMAIL / _PASSWORD	development account	First admin account. **Change before a pilot.**
FROSTMIND_CORS_ORIGINS		Allowed browser origins
FROSTMIND_OLLAMA_URL	http://localhost:11434/api/chat	LLM endpoint
FROSTMIND_OLLAMA_MODEL	gemma-4 E4B QAT (4-bit)	Preferred model
FROSTMIND_OLLAMA_TIMEOUT_SECONDS	45	LLM timeout

Table 17. *Backend configuration*

10.7 Flashing the firmware

```
arduino-cli board list                  # find the port, e.g. COM5
set FQBN=esp32:esp32:esp32s3:CDCOnBoot=cdc
arduino-cli compile --fqbn %FQBN% firmware/frostmind_main
arduino-cli upload -p COM5 --fqbn %FQBN% firmware/frostmind_main
.venv\Scripts\python scripts/esp_console.py   # live console
```

> **Note**
> `CDCOnBoot=cdc` is required. Without it, flashing succeeds but the sketch's serial output goes to unused UART pins and the console stays silent after the boot banner.

10.8 Troubleshooting

Symptom	Fix
The box does not appear in the phone's Bluetooth picker	Turn on Bluetooth and Location; allow Chrome 'Nearby devices'; close other FROSTMIND tabs (one connection at a time); power-cycle the box.
The phone shows certificate or network errors	Accept the certificate at both :8000/health and :5500. Check that the phone and the laptop are on the same Wi-Fi.
The admin panel loads but buttons do nothing from another device	Restart the launcher on the current network (it allows the current addresses at start).
Temperature and humidity read 0.00	A loose ground or power wire: check the DHT's 3V3 and the ground rail. (Found on the bench: a probe sketch showed the sensor itself was fine.)
Light strip shows wrong colours	Run g: the first colour must be red. If not, the strip isn't GRB.
Last LEDs of the strip stay dark	Set `STATUS_LED_COUNT` to the strip's LED count and reflash.
The IR LED never flickers on camera	The long leg must be in pad D5 (not D6).
No AI explanations	Check Ollama is running; the AI Models page and `/health` show the LLM status, and Test LLM gives a real reply.

Table 18. *Troubleshooting*

10.9 Path to production

The laptop deployment is right for a pilot and a demonstration. For a real fleet: switch to PostgreSQL (one connection string, since all queries go through the ORM); serve the API on a real domain with a Let's Encrypt certificate; containerise the backend and admin panel with Docker; move LLM inference to a small GPU server; and add log aggregation and uptime alerting. Section 14 lists these on the roadmap.

11. Usability

11.1 Users and their goals

User	Goal	Where
Driver	Set the truck up in a few minutes and not have to think about it again	Phone app and the light strip on the box
Fleet manager	See every truck, act on problems, prove compliance	Admin panel
Technician	Install and check a box	Wiring guide, serial console, test keys

Table 19. *Users*

11.2 Design principles applied

- **Nothing to type:** the device ID comes from the Bluetooth connection and the server address from the page URL. The driver never enters an IP or an ID.
- **Guided and verified:** a 6-step wizard that refuses to finish on faith. It waits for real readings, and the Identify button proves that the phone can reach the box.
- **Errors as instructions:** Bluetooth failures explain what to do next ('Turn on Bluetooth and Location, then tap Pair again'), not the browser's raw error.
- **Plain-language AI:** each alert's explanation ends with a *Do:* action a driver can follow.
- **Glanceable status:** the light strip follows the trip and shows one look at a time. It stays calm when nobody is connected, and on idle it shows just the shelf life.
- **Consistent meaning:** the same colours mean the same thing on the strip, the phone and the admin panel: mint is good, amber is a warning, coral is a problem, and the blue-violet-pink gradient is the AI.
- **Forgiving:** an offline queue, auto-reconnect, a wake lock, and a reset option for a new trip.
- **Accessible:** status is never shown by colour alone (the phone states in words what the strip shows), with large touch targets, high-contrast dark UI and capped LED brightness.

11.3 Improvements found through real use

The system was used end to end on real hardware, and every problem found was fixed and documented:

Problem found	Fix
The box never appeared in Chrome's Bluetooth picker	Advertise the service UUID in the advertisement, with a name-prefix filter as fallback
Phone commands silently ignored by the box	The firmware reassembles 20-byte Bluetooth chunks into whole lines
The phone app was only a monitor; no multi-product loads, no AI visible	Added the 'What are you shipping?' step with the AI load check, and the AI panel with explanations
Admin panel dead when opened from another device	Allow the laptop's current addresses; the API base follows the page host
Hard-coded IP broke on a new network	No-IP-lock design (Section 10.4)
Certificate warning after every restart	Reuse the certificate while it is still valid
Alert times 5.5 hours off on the phone	Server timestamps are parsed as UTC

Problem found	Fix
Two Bluetooth commands at once collided	Commands are queued
Wiring by D-label vs GPIO number caused dead parts	Every pin table now lists the XIAO pad first
Light strip flickered between an animation and red; errors shown with no phone connected	Workflow engine: one look at a time, problems only while live

Table 20. *Usability issues found and fixed*

12. Testing and quality assurance

Area	How it is tested	Result
Backend	pytest: ingest and alerts, shelf life, thresholds, device auth, GPS, error logging, LLM orchestrator, Model 1 seam, analytics, products, planner, mixed loads	**72 passed**
Admin panel	ESLint, TypeScript (tsc), `next build`	Clean
Phone app	`node --check` on every module; browser checks with a simulated Bluetooth device	Clean
Firmware	`arduino-cli compile --warnings all`	0 warnings
Hardware	Bench checks on the real board with the serial console and probe sketches	DHT22 24.5 °C, door open/closed, strip GRB on 11 LEDs
End to end	Real backend, real local LLM, scripted shipments	See below

Table 21. *Test coverage*

End-to-end results on the running system:

- The Load Planner split apples, pears, lettuce and bananas into 3 compartments.
- An apples + bananas shipment was **rejected** with reasons (ethylene and temperature).
- An apples + pears shipment was created at **−0.9 °C** (method: ML, guarded by physics).
- Warm readings were flagged against the setpoint, with per-product shelf life live.
- The AI explanation reached the phone **7 seconds** after the alert; the LLM test answered in 1.6 s.

Honest status

Verified on hardware: the DHT22 on D8, the reed switch on D4, and the 11-LED strip on D3 in GRB order. **Still to verify:** the IR LED on D5 with the `x` test, and AC pairing against the target air conditioner.

13. Demonstration plan

13.1 Before the demo

1. Run `python start_frostmind.py` on the presentation laptop. Use the venue Wi-Fi or the laptop's hotspot; both work.
2. On the phone, accept the certificate at :8000/health and :5500, open the app and use Reset wizard.
3. Open the admin panel, then AI Models, then Test LLM, so the model is loaded before the audience is watching.
4. Power the box. Keep the sensor in a cool box with an ice pack, so the live view starts at the shelf-life bar rather than an alarm.
5. Have the serial console ready as a fallback.

13.2 Ten-minute script

Time	Show	Point to make
0:00	Architecture slide	Sense, act, explain; everything local
1:00	Power the box: frost bloom, then the blue beacon	No phone, so no alarms: nobody would see them
2:00	Wizard steps 1-2: pair (handshake on the strip), register	Nothing typed; the device ID comes over Bluetooth
3:00	Step 3: apples, pears, lettuce, bananas, then Check load (AI)	Model 5 splits them and explains why; Model 4 gives −0.9 °C
4:30	Create the apples + pears shipment; the strip ripples and ticks 'product'	The box knows the workflow
5:00	Step 5: live data + Flash the lights (rainbow)	Both directions verified
5:30	Go live: sweep, then the shelf-life bar	On idle, just the shelf life
6:00	Take the sensor out / open the door	Coral heartbeat or amber door light; AI explanation on the phone within seconds
7:30	Admin panel: fleet, shipment page, alerts, Load Planner, AI Models	The manager's view, with measured model scores
9:00	Close: tests, deployment on any laptop, roadmap	Ready to run anywhere; honest about limits

Table 22. *Demonstration script*

13.3 Fallbacks

- **Bluetooth fails:** press `d` in the serial console to play the whole light-strip workflow, and use the admin panel with the shipment already created.
- **The LLM is slow:** the phone shows 'AI explanation being written…' and the deterministic alert is already there; explain the 45-second background design.
- **Wi-Fi blocks devices from seeing each other:** use the laptop's hotspot. The launcher finds the new address by itself.
- **The AC cannot be reached:** show the IR LED flashing through a phone camera with `x`.

14. Future scope

Horizon	Feature	Value
Next	Train Model 1 on FROSTMIND's own logged trips and switch it on	Learned anomaly detection on the live stream; the seam is ready
Next	Real-time push (WebSockets) instead of 10-second polling	Alerts appear instantly
Next	Map view of trucks (GPS is already recorded with every reading)	Fleet-wide situational awareness
Mid	INA219 current sensor, then Model 2 root-cause classification	Tell a compressor fault from an open door or a power cut
Mid	Learned correction layer on top of Model 3 using delivery-quality outcomes	Shelf life calibrated to real products
Mid	Fine-tune Model 4 and 5 with real delivery outcomes	From expert labels to measured outcomes
Mid	Over-the-air firmware updates (ESP32 OTA)	Update boxes without a laptop and cable
Mid	Cellular (LTE-M) or LoRaWAN module on the box	No dependence on the driver's phone
Long	Cloud deployment: PostgreSQL, TLS domain, Docker, GPU LLM server	Many fleets, many trucks
Long	Multi-tenant accounts and roles (driver, manager, auditor)	Sell to more than one company
Long	Native mobile app (React Native or Flutter)	iOS support and true background operation
Long	Integration with warehouse/ERP systems; GDP/FSMA-style certified reports	Fits into real logistics workflows
Long	Rugged enclosure, battery backup, DS18B20 probes per compartment	Field-ready hardware

Table 23. *Roadmap*

15. Maintenance strategy

FROSTMIND is built so that a small team can keep it healthy: every part reports its own status, models can be retrained with one command, and nothing breaks silently.

15.1 Routine tasks

Area	Task	When	How
Code	Run all checks before merging	Every change	pytest; npm run lint + build; arduino-cli compile --warnings all
Code	Review on a pull request into master	Every feature	GitHub PR
Models	Rebuild the catalog and retrain Models 4 and 5	When the catalog changes, else quarterly	scripts/train_all.py; compare *_metrics.json
Models	Check Model 4's σ against real truck data	Monthly	Planner uses each truck's own σ when available
Data	Back up the database	Weekly and before upgrades	Timestamped copy of frostmind.db
LLM	Check the LLM is reachable	Daily in use	AI Models page, Test LLM, /health
Hardware	Compare the DHT22 with a reference thermometer	Monthly	Replace if off by more than 0.5 °C
Hardware	Check wiring, strip and IR	Monthly and after moving a box	Console keys g, d, x
Security	Rotate the JWT secret and admin password	Before a pilot, then every 90 days	Environment variables
Security	Renew the HTTPS certificate	Automatic	Launcher regenerates it under 30 days left
Dependencies	Update libraries, then run the full test suite	Quarterly	Pinned versions (esp32 core 3.3.11, IRremoteESP8266 2.9.0)
Docs	Update docs with the change they describe	Every change	Same commit as the code

Table 24. *Maintenance schedule*

15.2 Model governance

- **Every model ships with its metrics:** each `.pkl` has a `*_metrics.json` with its scores, threshold and features, shown on the AI Models page.
- **Promotion gate:** a retrained model replaces the old one only if its product-disjoint scores are at least as good (e.g. Model 5 ROC-AUC ≥ 0.90, Model 4 within ±0.5 °C ≥ 90 %).
- **Guarded serving:** Model 4's answer is used only if the physics optimiser agrees to 0.5 °C, otherwise physics wins. If a `.pkl` is missing the backend falls back to physics and rules instead of crashing.

- **Safe activation:** Model 1 turns on only when a model trained on FROSTMIND's own data exists; until then it is a documented no-op.
- **LLM boundary:** the LLM can never act; any failure leaves the deterministic alert intact.

15.3 Monitoring and incident response

Signal	Where	Response
Backend health, model and LLM status	`/health`, AI Models page	Restart the launcher or Ollama
Device stopped reporting	Fleet page (online state), phone BLE tile	Check power and Bluetooth; phone reconnects automatically
IR path alive	'Last IR command sent' (heartbeat every 5 min)	If stale: check the IR LED with x
Readings waiting to sync	Phone 'Pending sync' tile	Check Wi-Fi; nothing is lost, the queue uploads later
Errors	Structured backend logs	Each failure logged with context

Table 25. *Monitoring signals*

15.4 Data and schema changes

Schema changes are additive (`ensure_columns()` adds new nullable columns on start-up), so upgrading never deletes data. Back up the database before any upgrade. Moving to PostgreSQL is a connection-string change because every query goes through the ORM.

15.5 Roles

Role	Responsible for
Firmware owner	Firmware changes, flashing, hardware checks, wiring docs
Backend and AI owner	API, database, model retraining and promotion, LLM setup
Frontend owner	Phone app and admin panel, usability fixes
Release owner (rotating)	Runs the checks, merges the PR, updates documentation, backs up the database

Table 26. *Maintenance roles*

16. Limitations

- Android only: iOS Safari does not support Web Bluetooth.
- The phone must keep the app in the foreground; browsers cannot hold Bluetooth in the background.
- The LAN deployment uses a self-signed certificate and a single SQLite database, and the setup endpoints are open on the local network.
- Model 1 is validated but not live; Model 2 is not built.
- Model 3's constants are illustrative literature values, not certified for a specific product.
- Models 4 and 5 are trained on expert labels and simulated loads; non-produce pairs are handled by rules and marked as extrapolated.
- IR is transmit-only: the box cannot confirm the AC received a command (mitigated by the heartbeat).
- Not certified for regulated pharmaceutical decisions.

17. Conclusion

FROSTMIND is a complete, working cold-chain system built on low-cost, off-the-shelf hardware. It senses every 2 seconds, corrects the cooling unit on its own, measures shelf life per product, recommends temperatures for mixed loads, decides which products can travel together, and explains every incident in plain English, all on one laptop with no cloud account.

For Phase 7 the system is deployable on any laptop with one command and usable by a driver through a guided setup and a glanceable light strip. It is documented and tested (72 automated tests), with a clear plan for demonstration, growth and maintenance.

18. References

1. Gross, K. C., Wang, C. Y. and Saltveit, M. (eds.) (2016). *The Commercial Storage of Fruits, Vegetables, and Florist and Nursery Stocks*. USDA Agriculture Handbook 66.
2. McGregor, B. M. (1989). *Tropical Products Transport Handbook*. USDA Agriculture Handbook 668 (compatibility groups).
3. Engineering ToolBox. Fruits and Vegetables: Optimal Storage Conditions.
4. U.S. FDA (2022). *Food Code*.
5. U.S. CDC. *Vaccine Storage and Handling Toolkit*.
6. WHO / European Commission. *Good Distribution Practice* guidelines.
7. Labuza, T. P. (1984). Application of chemical kinetics to deterioration of foods. *Journal of Chemical Education*, 61(4).
8. Liu, F. T., Ting, K. M. and Zhou, Z.-H. (2008). Isolation Forest. *IEEE ICDM*.
9. Lavin, A. and Ahmad, S. (2015). Evaluating real-time anomaly detection algorithms: the Numenta Anomaly Benchmark. *IEEE ICMLA*.
10. Abhiraman et al. (2023). Vaccine-refrigeration fault simulation with CNNs. *International Journal of Refrigeration*.
11. Brélaz, D. (1979). New methods to color the vertices of a graph (DSatur). *Communications of the ACM*, 22(4).
12. Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *JMLR*, 12.
13. Espressif Systems. ESP32-S3 Technical Reference Manual; Seeed Studio XIAO ESP32S3 wiki.
14. W3C Web Bluetooth Community Group. *Web Bluetooth* specification.
15. FastAPI, Next.js, Ollama, Gemma and IRremoteESP8266 project documentation.

Appendix A. Repository structure

Path	Contents
start_frostmind.py	One-command launcher (servers, certificate, addresses)
firmware/frostmind_main/	Device firmware and its README (BLE contract, light strip, flashing)
phone-bridge/	Phone PWA: js/ (wizard, dashboard, ble, api, sync, idb, geo), css/, service worker, serve_https.py
backend/app/	FastAPI app: routers/, models.py, schemas.py, ml.py, load_planner.py, llm_orchestrator.py
backend/tests/	72 automated tests
web/src/	Admin panel: app/ (pages), components/, lib/
scripts/	Data fetching, cleaning, training (train_all.py), esp_console.py
data/	Datasets and the product catalog with its cleaning report
models/	Trained models and their metrics (the .pkl files are built locally)
docs/	Figures and the document converter
Phase7_Submission/	This document, the presentation, figures and their generators

Table 27. *Repository structure*

Appendix B. API endpoints

Method	Path	Auth	Purpose
POST	/auth/login · GET /auth/me	- · JWT	Admin login and profile
POST	/devices/register	open	Register a device, get its key
GET/PATCH	/devices, /devices/{id}	open	List and update devices
POST/GET	/devices/{id}/pairing/start, /status	open	AC pairing state from the phone
GET/POST/PATCH	/trucks, /trucks/{id}	open	Trucks
POST	/shipments	open	Create a shipment (category or products, Model 5 check, Model 4 setpoint)
GET/PATCH	/shipments, /shipments/{id}	JWT	Fleet list, detail, status
GET	/shipments/{id}/readings, /alerts	JWT	History
GET	/shipments/{id}/live	Device key	Phone AI feed
GET / POST	/shipments/{id}/report · /compliance_report	JWT	CSV export · LLM report
POST	/ingest	Device key	Upload readings
GET/PATCH	/alerts, /alerts/{id}	JWT	Alerts and acknowledgement
GET	/analytics/fleet	JWT	Fleet trends
GET	/products	open	155-product catalog
POST	/planner/plan	open	Compartments, setpoints, compatibility
GET · POST	/ai/status · /ai/llm/test	open · JWT	AI component status · LLM round trip
GET	/health	open	Server, model and LLM status

Table 28. *API endpoints (interactive docs at /docs)*

Appendix C. Command cheat sheet

```
# run everything
.venv\Scripts\python start_frostmind.py

# tests and checks
cd backend && ..\.venv\Scripts\python -m pytest
cd web && npm run lint && npm run build
arduino-cli compile --warnings all --fqbn esp32:esp32:esp32s3:CDCOnBoot=cdc ^
    firmware/frostmind_main

# retrain models
.venv\Scripts\python scripts/train_all.py

# device console (keys: x IR test, g strip colours, d strip demo, a auto-pair)
.venv\Scripts\python scripts/esp_console.py
```

Appendix D. Glossary

Term	Meaning
Cold chain	The temperature-controlled supply chain for perishable goods
Reefer	A refrigerated truck or container unit
Excursion	A period outside the allowed temperature range
Setpoint	The temperature a cooling unit is told to hold
Q10	How many times faster a product decays for a 10 °C rise
Ethylene	A ripening gas some fruits emit that damages sensitive produce
BLE	Bluetooth Low Energy
PWA	Progressive Web App: a web page that installs and works offline like an app
RMT	The ESP32's remote-control peripheral, used here to time the LED strip's data
MAE / ROC-AUC / MCC	Mean absolute error; area under the ROC curve; Matthews correlation coefficient
DSatur	A graph-colouring algorithm, used to split incompatible products into compartments
LLM	Large language model; here a small quantised model running locally

Table 29. *Glossary*